Day 4: SOC Lab Task Report

TCP Connection Capture: Handshake and Teardown Analysis

Field	Details
Intern Name	Amrita Pathak
Date	06 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	11:59 PM IST

Contents

1. Objective 3
2. Tools and Environment 3
3. Key Terms 3
4. Setting Up the Local Service and Capture 4
5. Generating the Connection 6
6. Stopping and Saving the Capture 8
7. The Three-Way Handshake 10
7.1 Packet 1: SYN 10
7.2 Packet 2: SYN, ACK 11
7.3 Packet 3: ACK 13
8. Data Transfer 15
9. Connection Teardown 15
9.1 Packet 10: FIN, ACK (server-initiated) 15
9.2 Packet 11: FIN, ACK (client response, combined) 17
9.3 Packet 12: ACK (final) 19
10. Connection Summary 21
11. Findings 21
11.1 The teardown closed in three packets, not the textbook four 21
11.2 Port usage matched the expected client-server pattern exactly 21
11.3 Capturing on loopback removed every source of unrelated noise 21
11.4 Sequence numbers displayed by Wireshark are relative, not the actual values sent 21
12. Conclusion 22

1. Objective

Capture a TCP connection to an authorized local service in Wireshark and locate the SYN, SYN-ACK, and ACK packets that open it. Record the sequence of events, the source and destination ports involved, and the packets that tear the connection down at the end.

2. Tools and Environment

Item	Detail
Capture tool	Wireshark 4.6.6
Capture host	Kali Linux 2026.2
Capture interface	lo (loopback)
Local service	Python's built-in http.server, port 8000
Connection generator	curl

The service used for this capture was a small web server started deliberately on the local machine (python3 -m http.server 8000) rather than any external or third-party host. This satisfies the authorized local service requirement unambiguously, since the service is self-hosted and self-administered, and it also produces a far cleaner capture than a real internet connection would: no DNS lookups, no background telemetry, no ARP or DHCP broadcasts competing for attention. The capture was taken on the loopback interface (lo), the internal path a machine uses to talk to itself, rather than eth0, since the client and server in this case are the same machine.

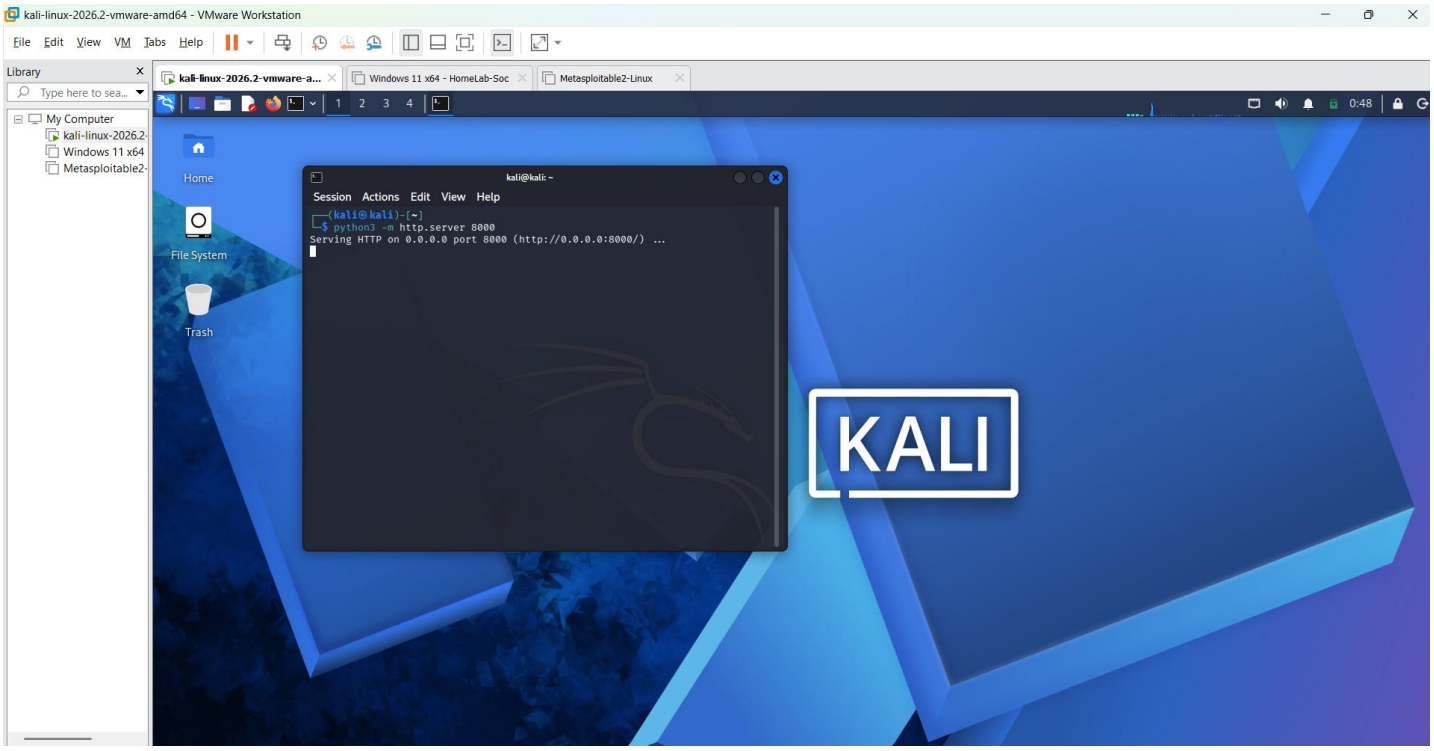
3. Key Terms

- Three-way handshake: the SYN, SYN-ACK, ACK exchange that opens a TCP connection. Each side confirms it can both send and receive before any real data moves.
- SYN (Synchronize): the first packet in the handshake, sent by the side initiating the connection, proposing an initial sequence number to count bytes from.
- SYN-ACK: the response from the listening side, agreeing to the connection and acknowledging the initiator's sequence number while proposing its own.
- ACK (Acknowledgment): a flag confirming receipt of data or of a connection-control packet. The number attached to it states the next byte the sender expects to receive.
- Sequence Number: a counter each side uses to track bytes it has sent, so the receiving side can detect loss, duplication, or reordering. Wireshark displays these as relative numbers starting near 0 for readability rather than the actual raw 32-bit value each side generates at random.
- FIN (Finish): a flag meaning the sender has no more data to send. Closing a TCP connection normally takes a FIN from each side, each acknowledged by the other.
- Ephemeral port: a temporary, randomly assigned port number a client uses for the outgoing side of a connection, as opposed to the fixed, well-known port a server listens on.
- Loopback interface (lo): the internal network path a machine uses to send traffic to itself, without ever reaching a physical network adapter.

4. Setting Up the Local Service and Capture

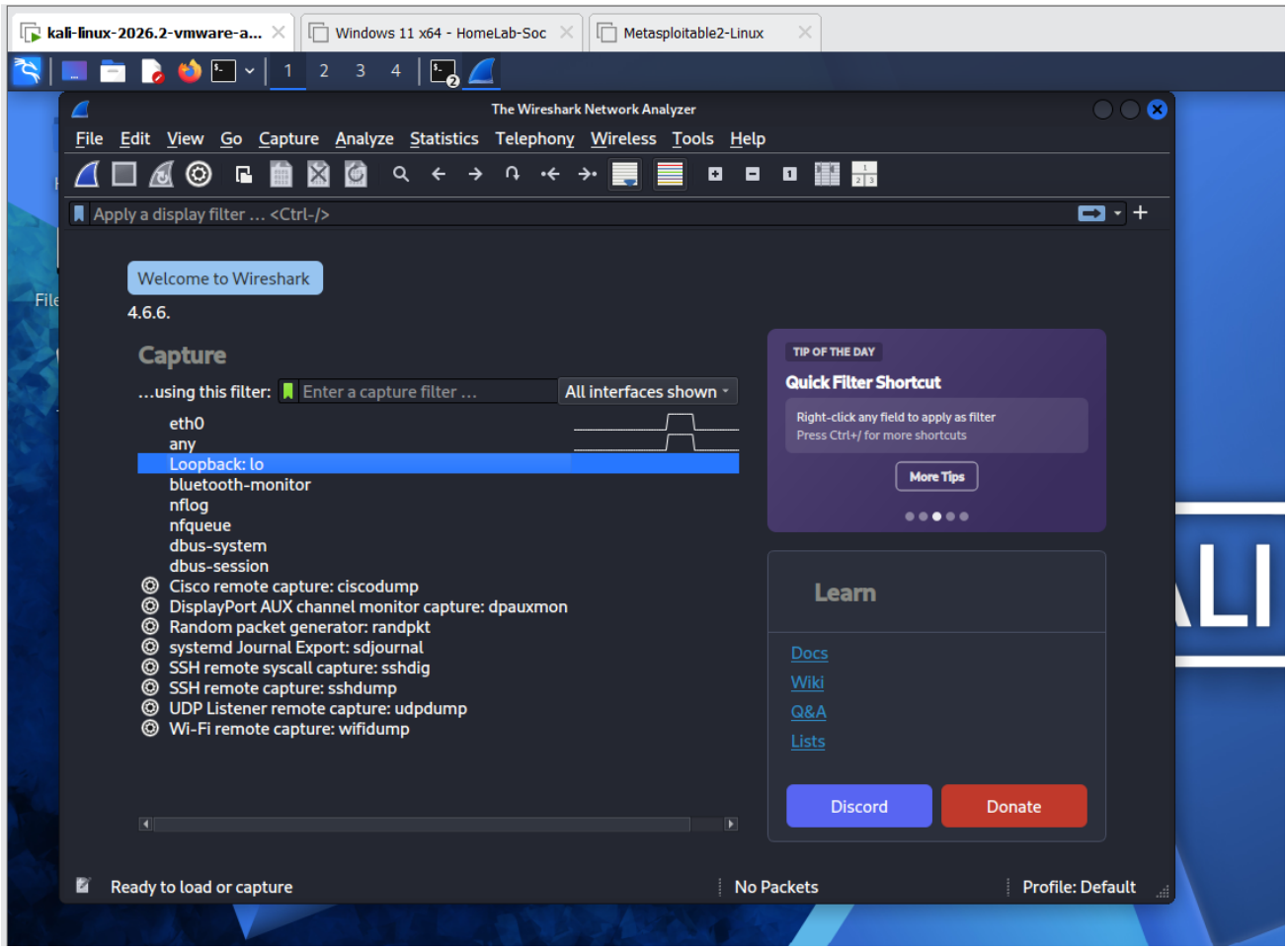
A local HTTP server was started on port 8000, deliberately chosen as the authorized target for this exercise since it runs entirely on the local machine under direct control.

```
python3 -m http.server 8000
```



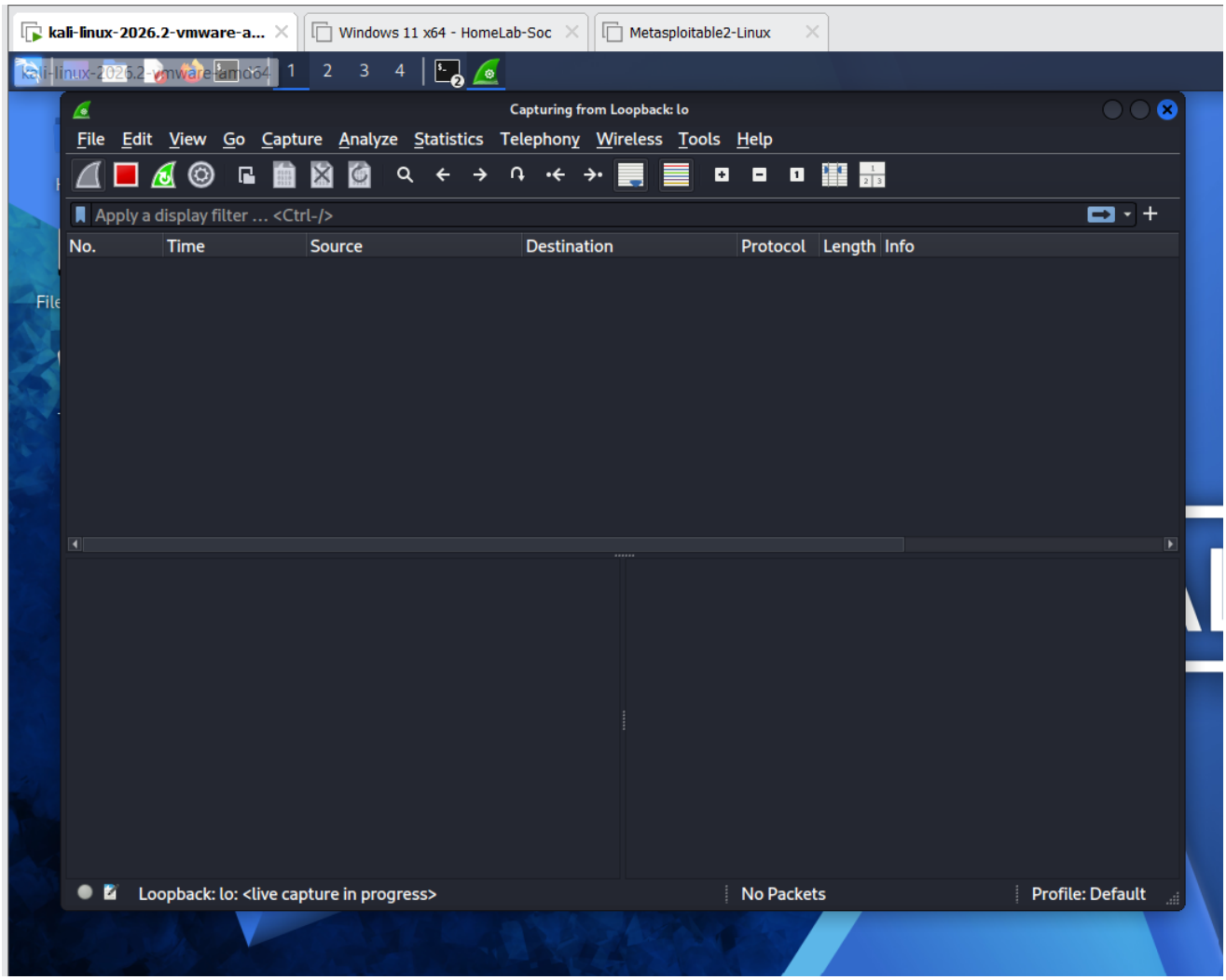
Local HTTP server started and listening on port 8000. This is the authorized local service the TCP connection will be made to.

Wireshark was then launched with sudo, since capturing raw traffic requires elevated privileges. Unlike the previous capture on eth0, this time the loopback interface was selected and confirmed active before any traffic was generated.



Wireshark's interface list, with Loopback: lo visible as an available capture target.

Capturing on lo rather than eth0 was a deliberate choice, not just a default. Traffic between a client and server on the same machine never touches a physical network adapter, so eth0 would show nothing at all for this connection, while lo carries every byte of it and nothing else, no broadcast traffic from other devices on the network to filter out.

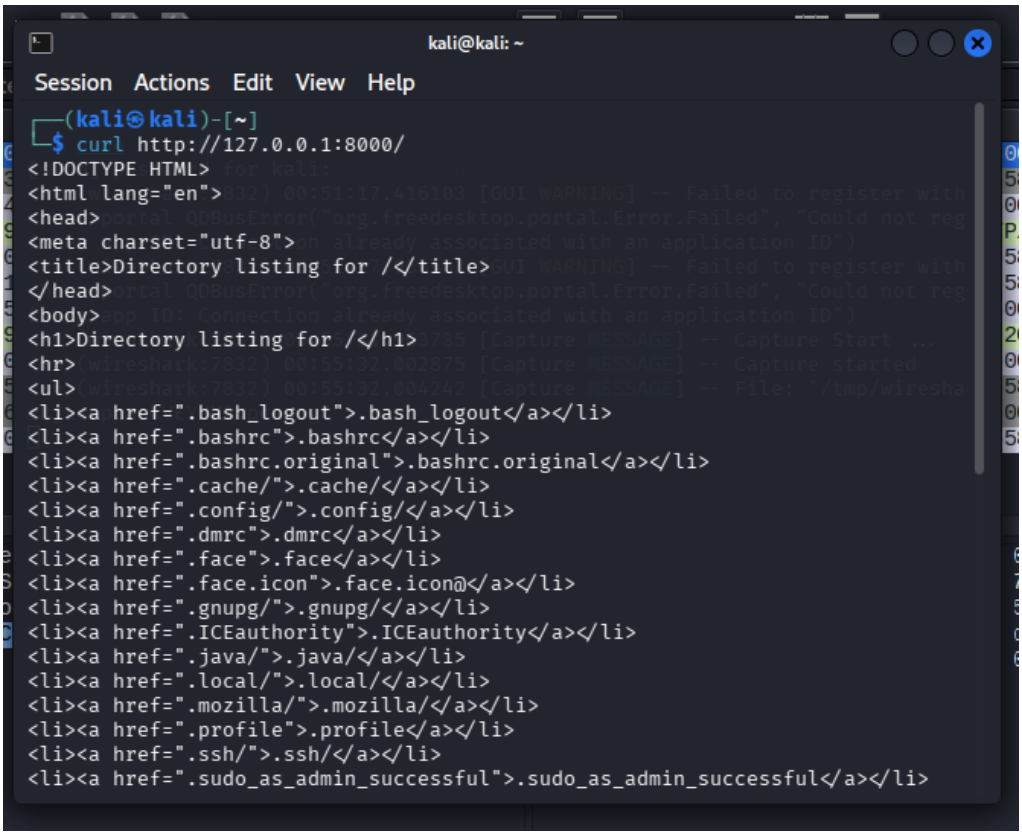


Capturing from Loopback: lo, confirmed in both the window title ("Capturing from Loopback: lo") and the status bar ("Loopback: lo: live capture in progress"). Zero packets at this point, since nothing has been sent over loopback yet.

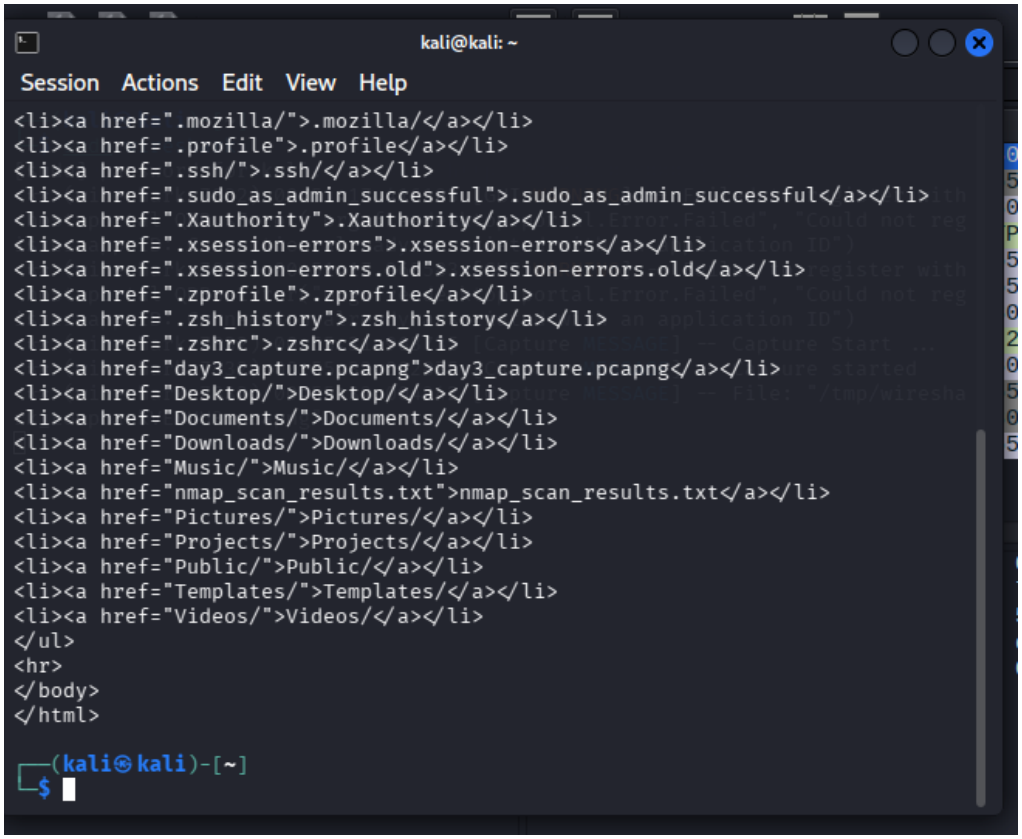
5. Generating the Connection

With the capture running, a single HTTP request was sent to the local server using curl, generating exactly one complete TCP connection to examine.

```
curl http://127.0.0.1:8000/
```

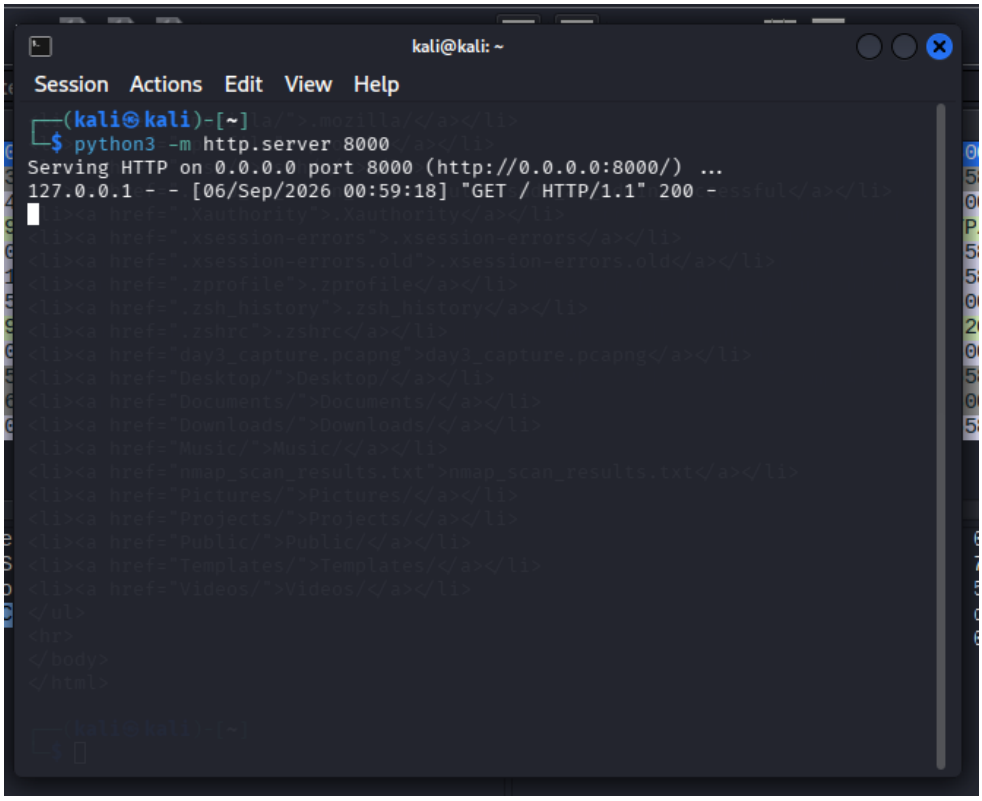


curl's request to the local server returned a full directory listing, confirming the connection completed successfully – part 1



curl's request to the local server returned a full directory listing, confirming the connection completed successfully – part 2

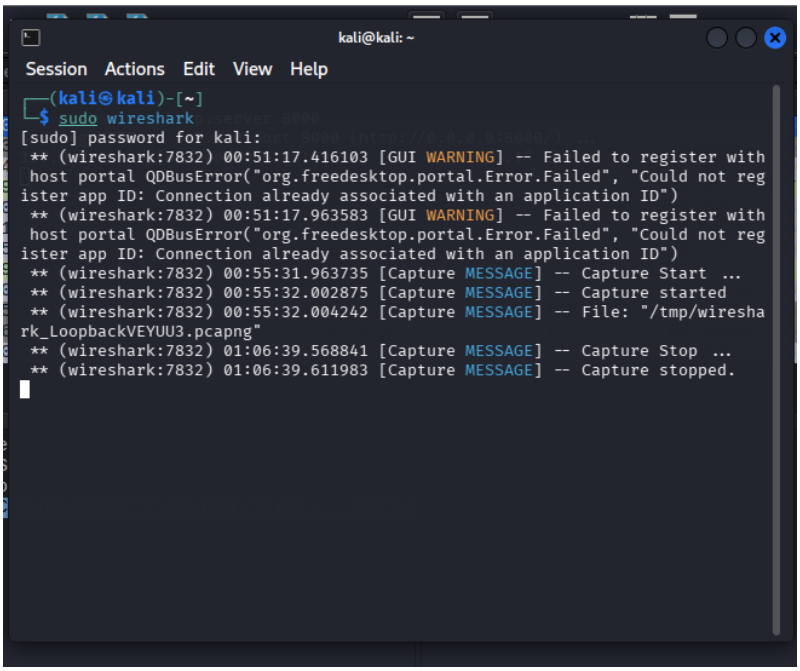
The server side was checked to confirm the request was received and answered.



The server's own log shows the incoming request: 127.0.0.1 - - [date] "GET / HTTP/1.1" 200 -, confirming a successful response was sent back.

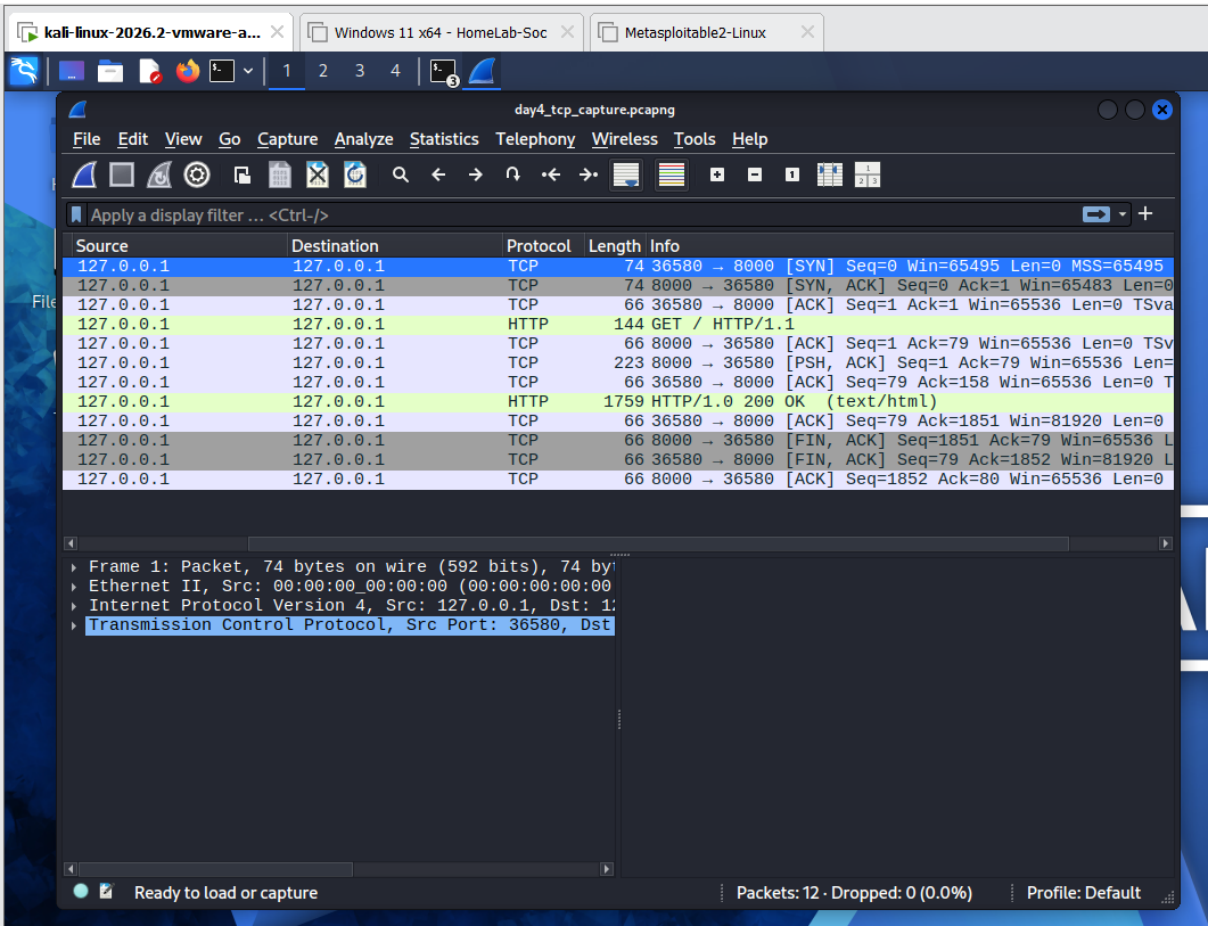
6. Stopping and Saving the Capture

Wireshark also logs its own lifecycle events to the terminal it was launched from, separately from the GUI capture window itself.



Terminal log showing Capture Start, Capture started, and later Capture Stop, Capture stopped, confirming the full lifecycle of the capture session.

The capture was stopped once the connection had fully completed, and saved as day4_tcp_capture.pcapng.



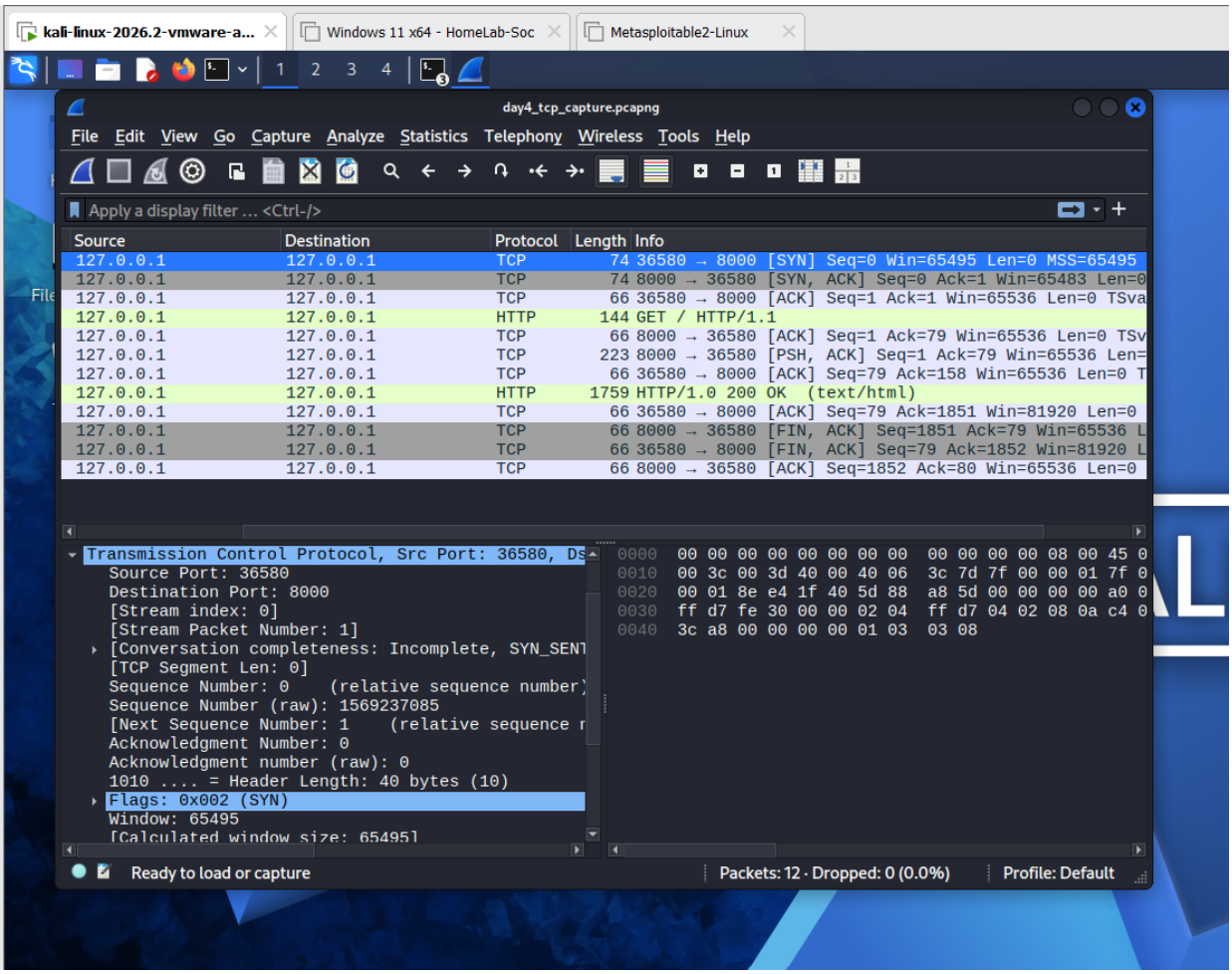
The saved capture: exactly 12 packets, covering the entire lifecycle of one TCP connection from open to close, with no unrelated traffic mixed in.

This is a strikingly clean result compared to a capture on a live network interface. All 12 packets belong to the same single connection, which makes it possible to read the entire sequence of events directly from the packet list without filtering anything out.

7. The Three-Way Handshake

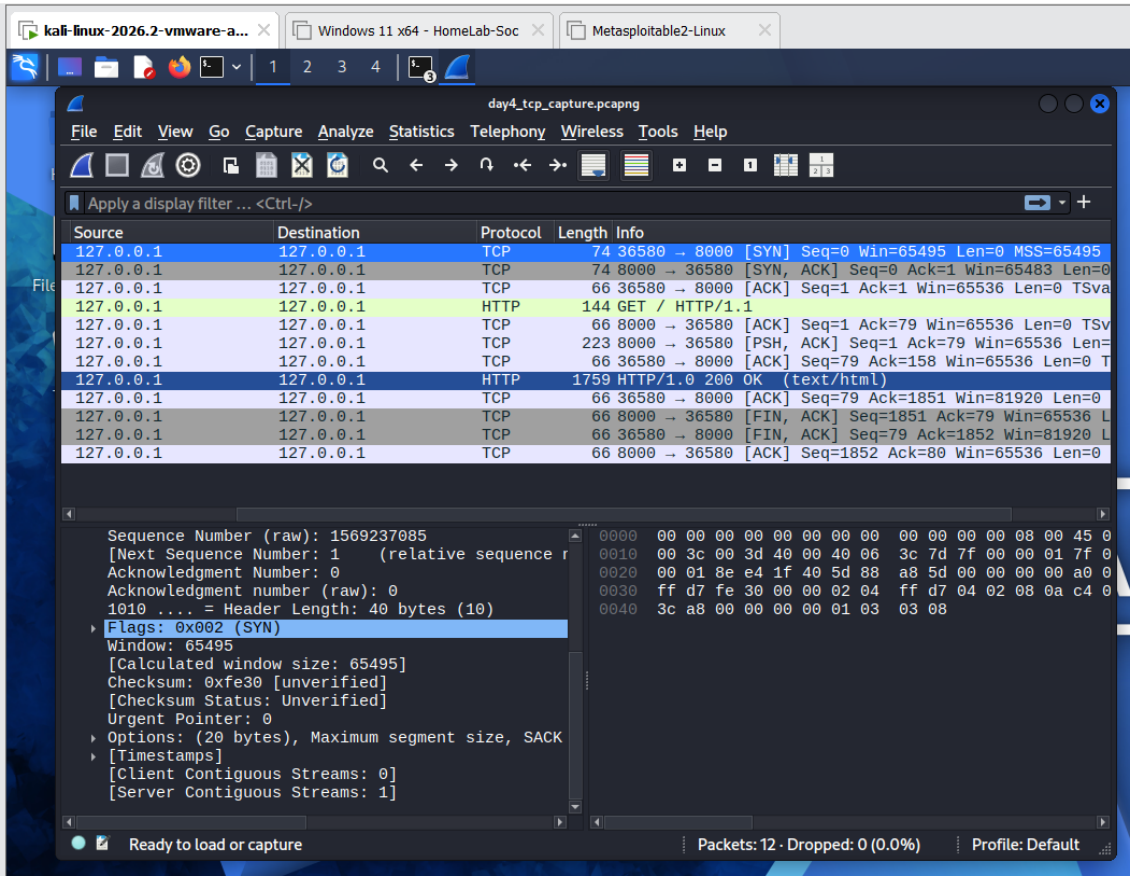
The first three packets in the capture form the complete handshake that opens the connection. Each was selected individually and its TCP header expanded to confirm the exact field values, not just the summary line.

7.1 Packet 1: SYN



Packet 1: Source Port 36580, Destination Port 8000, Sequence Number 0, Flags 0x002 (SYN). The client proposes to open a connection.

This packet is sent from the client's ephemeral port 36580 to the server's fixed port 8000, proposing sequence number 0 as the starting point for the bytes it will send.

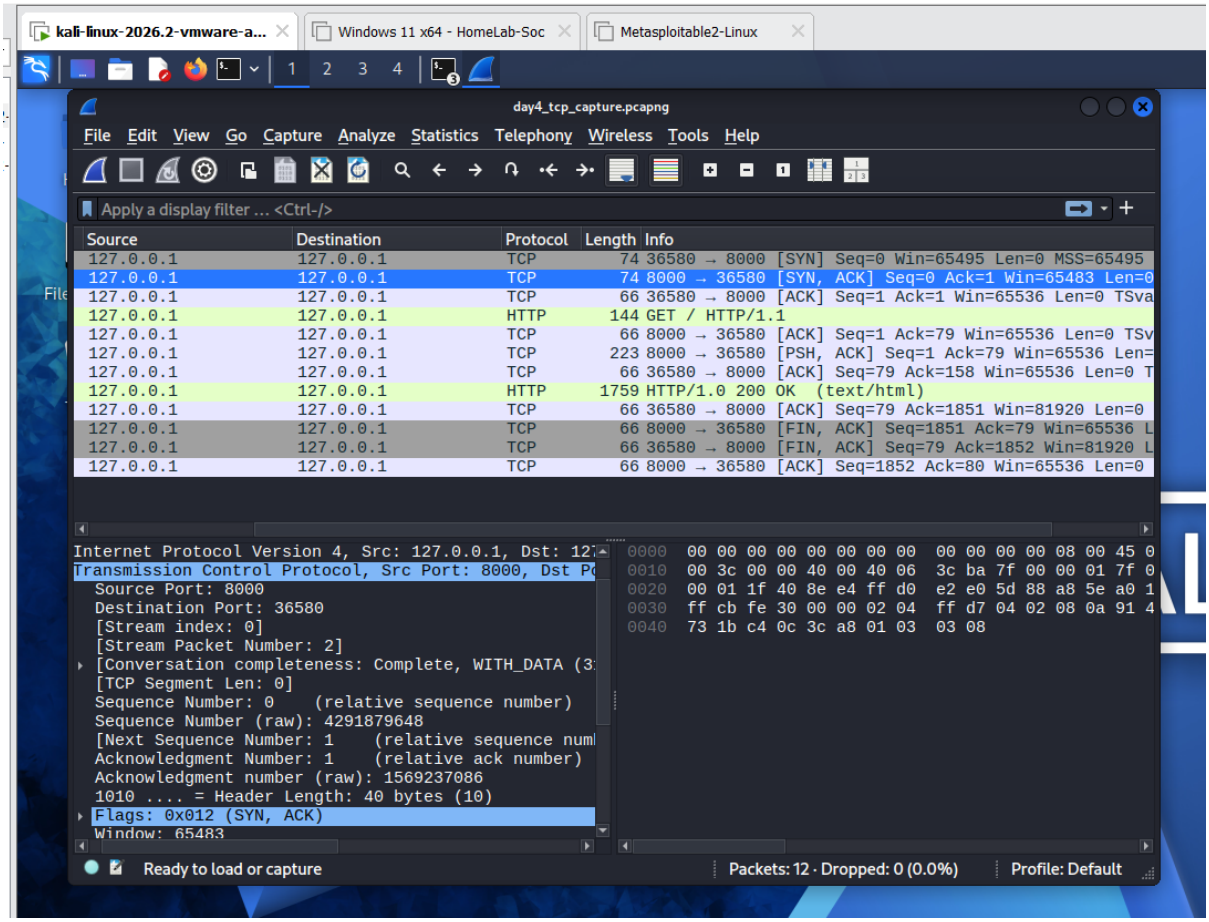


Continued detail for packet 1: checksum, TCP options (Maximum Segment Size and SACK permitted), and stream classification confirming this is the first packet of a new client-initiated stream.

Three fields here carry real analytical weight beyond just confirming the SYN. First, the checksum shows as "unverified" rather than pass or fail, this is normal and not a red flag: modern network interfaces calculate TCP checksums in hardware at the point a packet actually leaves the machine, which happens after Wireshark has already captured it, so Wireshark has nothing to verify against.

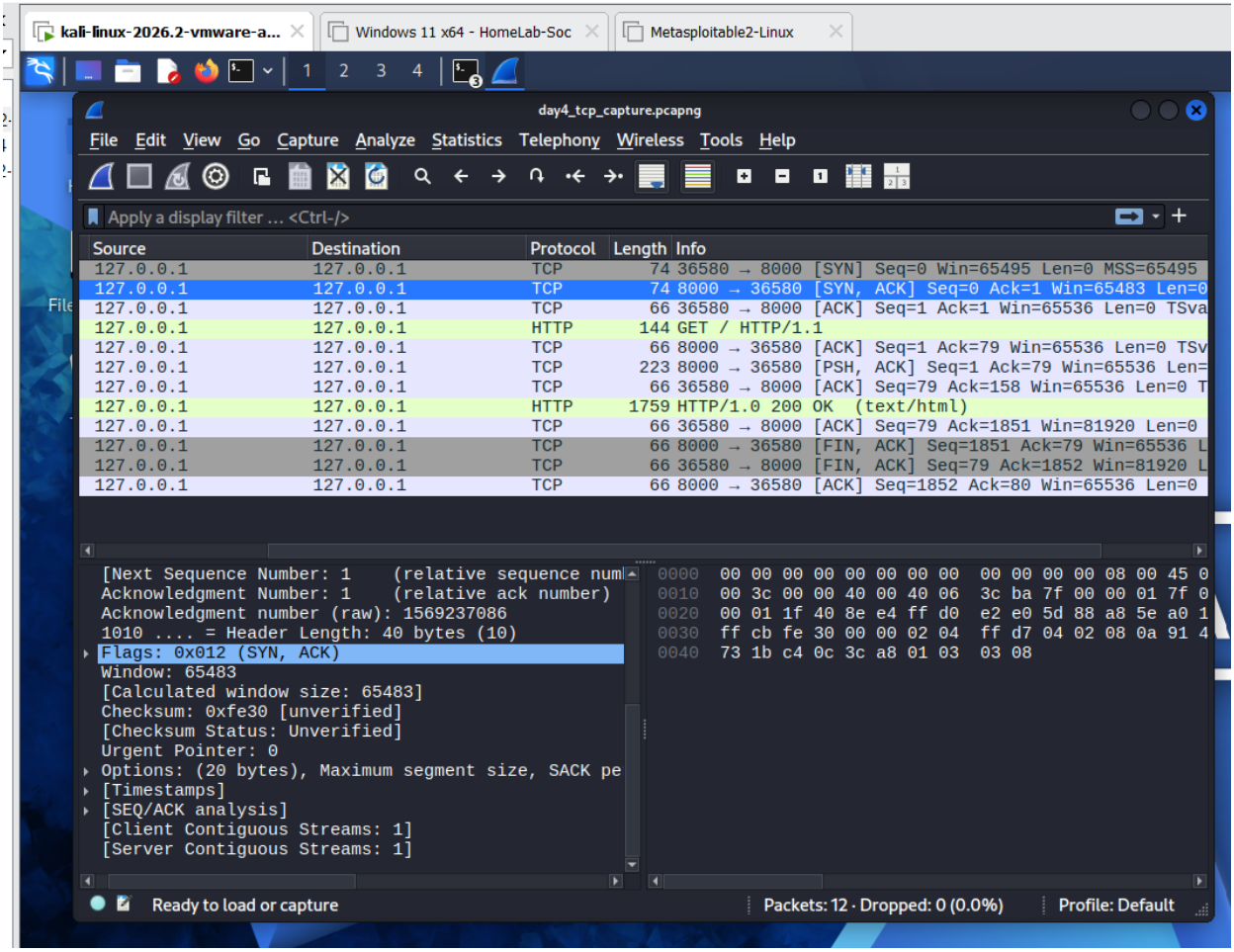
Second, the Options field shows Maximum Segment Size and SACK permitted being negotiated before any data flows: MSS tells the other side the largest chunk of data this side is willing to receive in one piece, and SACK permitted (Selective Acknowledgment) means that if packets later arrive out of order or with gaps, the receiver can acknowledge the specific ranges it did receive rather than forcing a full retransmission of everything after the gap. Third, "Client Contiguous Streams: 0, Server Contiguous Streams: 1" is Wireshark's own internal bookkeeping confirming this SYN is the very first packet of a brand-new TCP stream it has not seen before, exactly as expected for a connection's opening packet.

7.2 Packet 2: SYN, ACK



Packet 2: Source Port 8000, Destination Port 36580, Sequence Number 0, Acknowledgment Number 1, Flags 0x012 (SYN, ACK). The server accepts and acknowledges.

The server replies from port 8000 back to the client's port 36580, acknowledging the client's SYN (Ack=1, meaning it is ready for the client's next byte) while proposing its own starting sequence number.

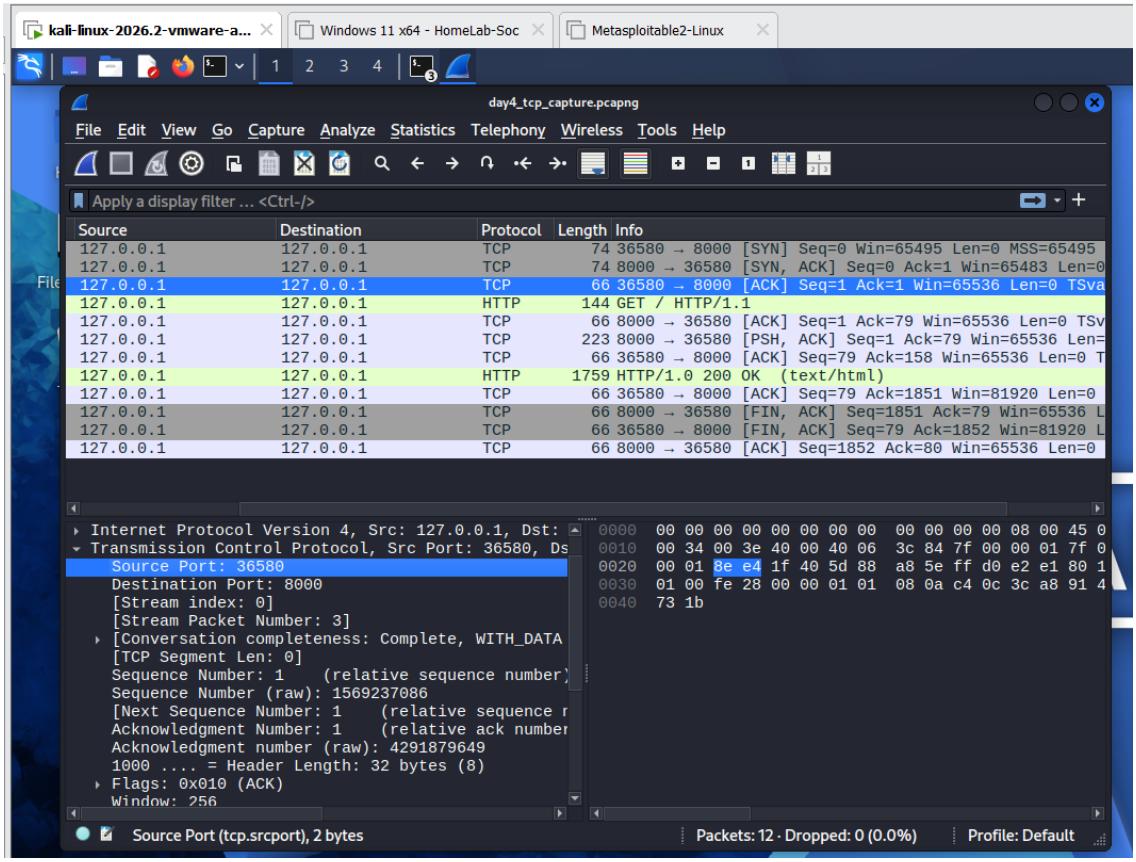


Continued detail for packet 2: the raw acknowledgment number (1569237086) matches the client's raw initial sequence number from packet 1 plus one, confirming the server correctly acknowledged the client's SYN at the byte level, not just in Wireshark's simplified relative view.

This is the single most important verification step in reading a handshake correctly: trusting the relative numbers Wireshark displays (Seq=0, Ack=1) is fine for a quick read, but confirming the underlying raw arithmetic is what separates a genuine check from an assumption.

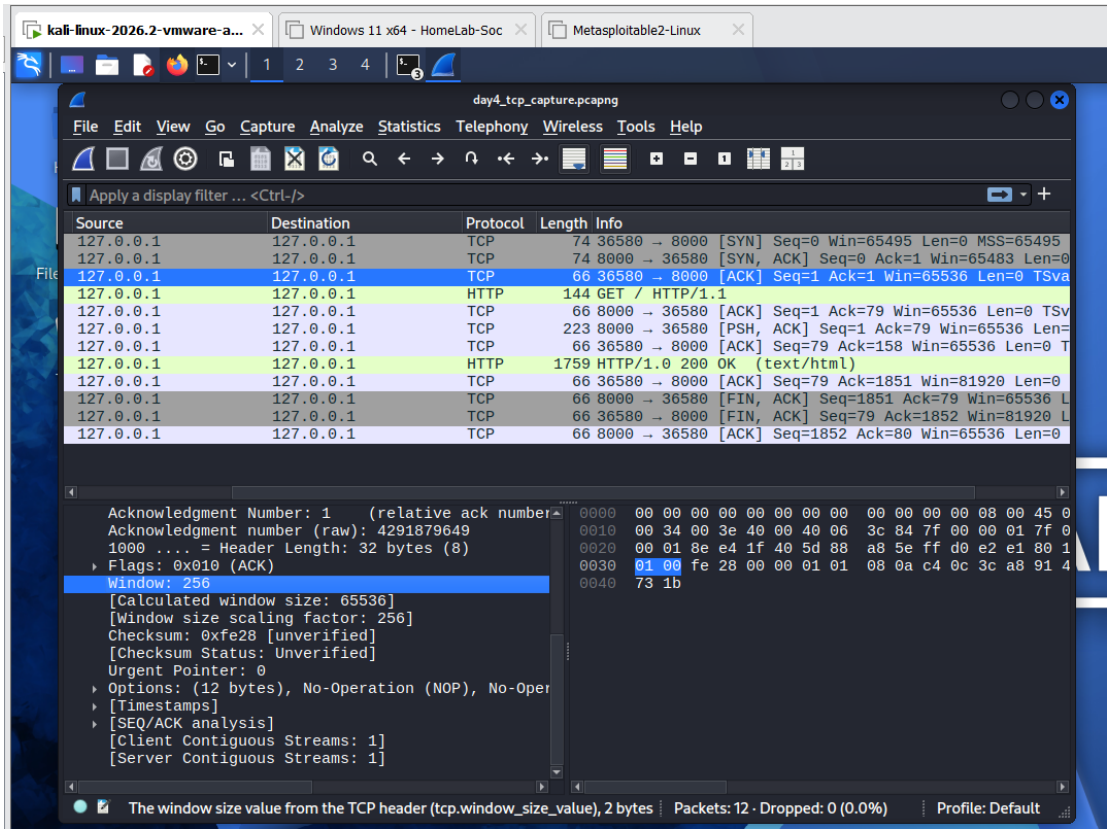
The client's raw starting sequence number in packet 1 was 1569237085; the server's acknowledgment number here is 1569237086, exactly one higher, which is the correct behavior since a SYN consumes one sequence number even though it carries no actual data. The server also negotiates its own Maximum Segment Size and confirms SACK support back to the client in the same Options field, meaning both sides have now agreed on these parameters before a single byte of the HTTP request has moved.

7.3 Packet 3: ACK



Packet 3: Source Port 36580, Destination Port 8000, Sequence Number 1, Acknowledgment Number 1, Flags 0x010 (ACK). The client confirms, and the connection is now open.

The client acknowledges the server's SYN-ACK. With this packet, the three-way handshake is complete and the connection is fully established in both directions.



Continued detail for packet 3: a window scaling factor of 256 is applied, meaning the advertised window value is multiplied by that factor to give the true 65536-byte receive window, a detail negotiated during the handshake and only visible when this field is expanded.

The raw Window field in a TCP header is only 16 bits wide, capping its raw value at 65535, which was a serious limitation once networks became fast enough to keep far more data in flight than that. Window scaling, negotiated as an option during the handshake and not visible on ordinary data packets afterward, fixes this by having both sides agree on a multiplier applied to every window value for the rest of the connection.

Here, the raw field shows 256, and with the negotiated factor of 256 applied, the true advertised receive window is 65536 bytes, meaning the client is telling the server it can hold a full 64KB of unacknowledged data in its receive buffer before the server needs to pause and wait for an acknowledgment. This packet also carries the SEQ/ACK analysis Wireshark performs automatically, confirming this ACK correctly and immediately follows the SYN-ACK it responds to with no gap or irregularity, which is the final confirmation that the handshake completed cleanly.

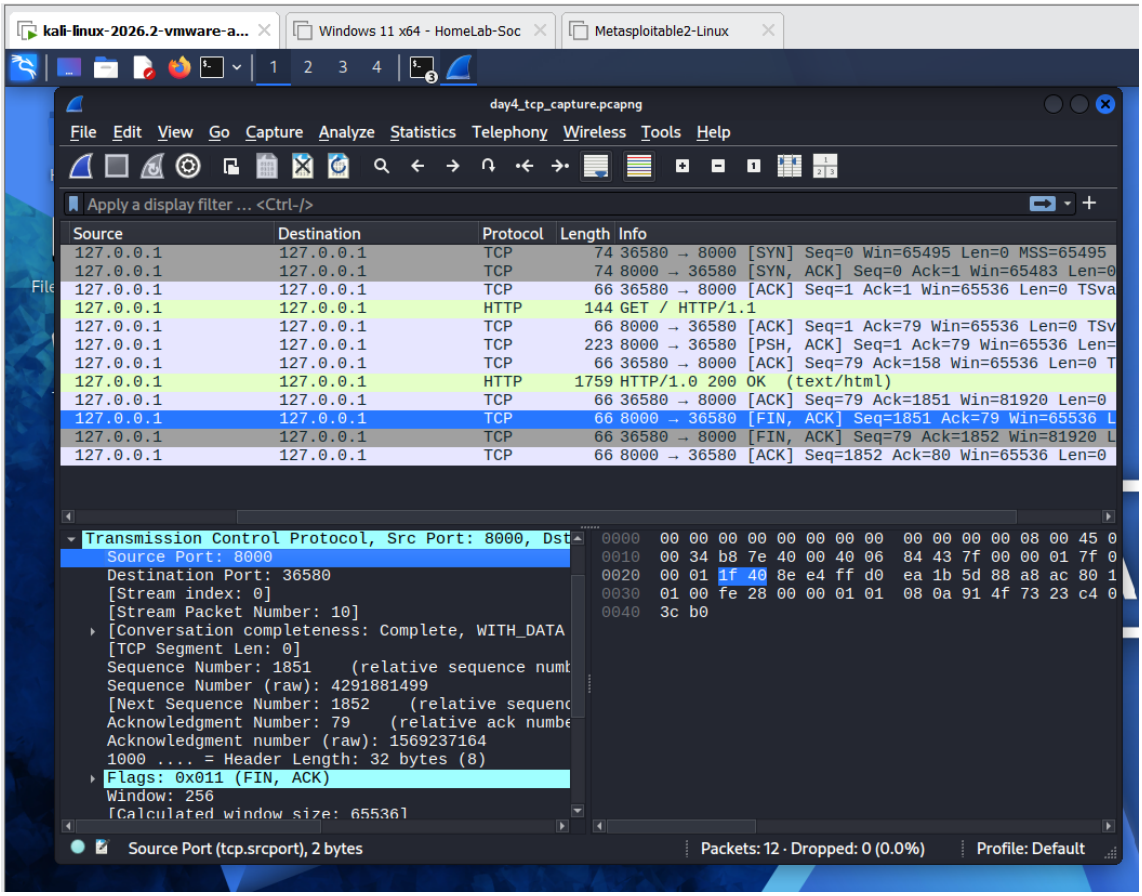
8. Data Transfer

Packets 4 through 9 carry the actual HTTP exchange: the client's GET request, the server's acknowledgment of it, the HTTP response itself, and the client's acknowledgment of that response. These packets sit on top of the now-open connection and were not individually expanded, since the task's focus is the connection's opening and closing behavior rather than its payload, but their presence in the capture (visible directly in the saved packet list in section 6) confirms the connection was used for real data before being closed.

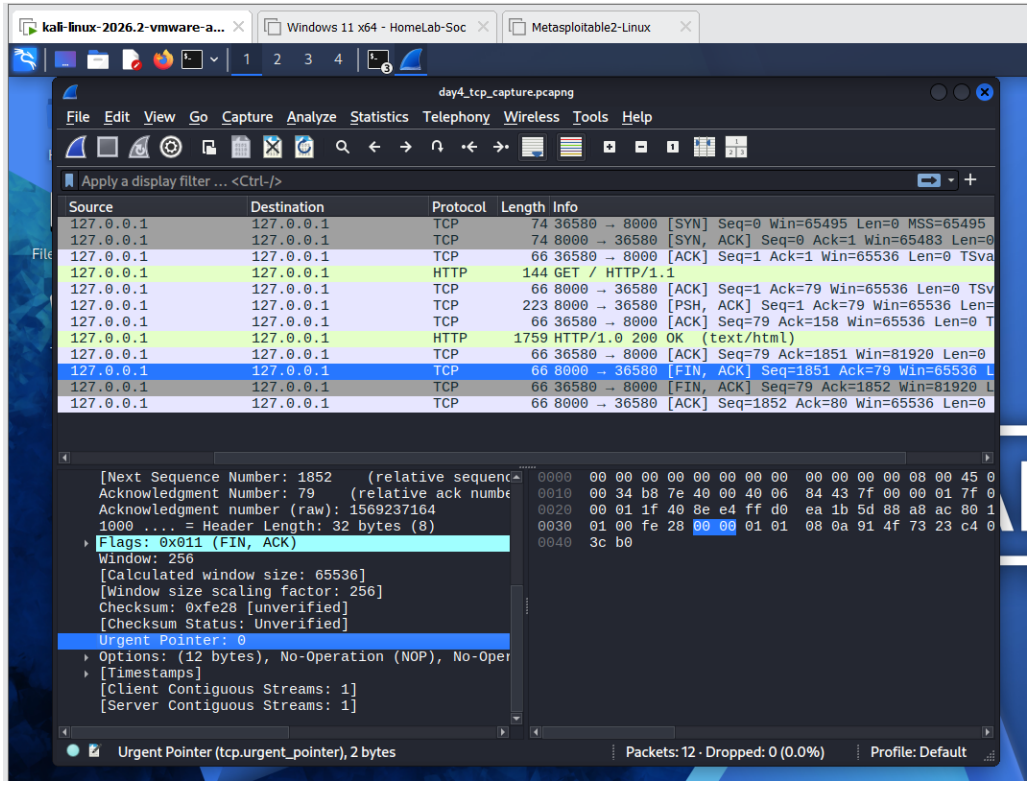
9. Connection Teardown

The final three packets close the connection. Each was expanded the same way as the handshake packets.

9.1 Packet 10: FIN, ACK (server-initiated)



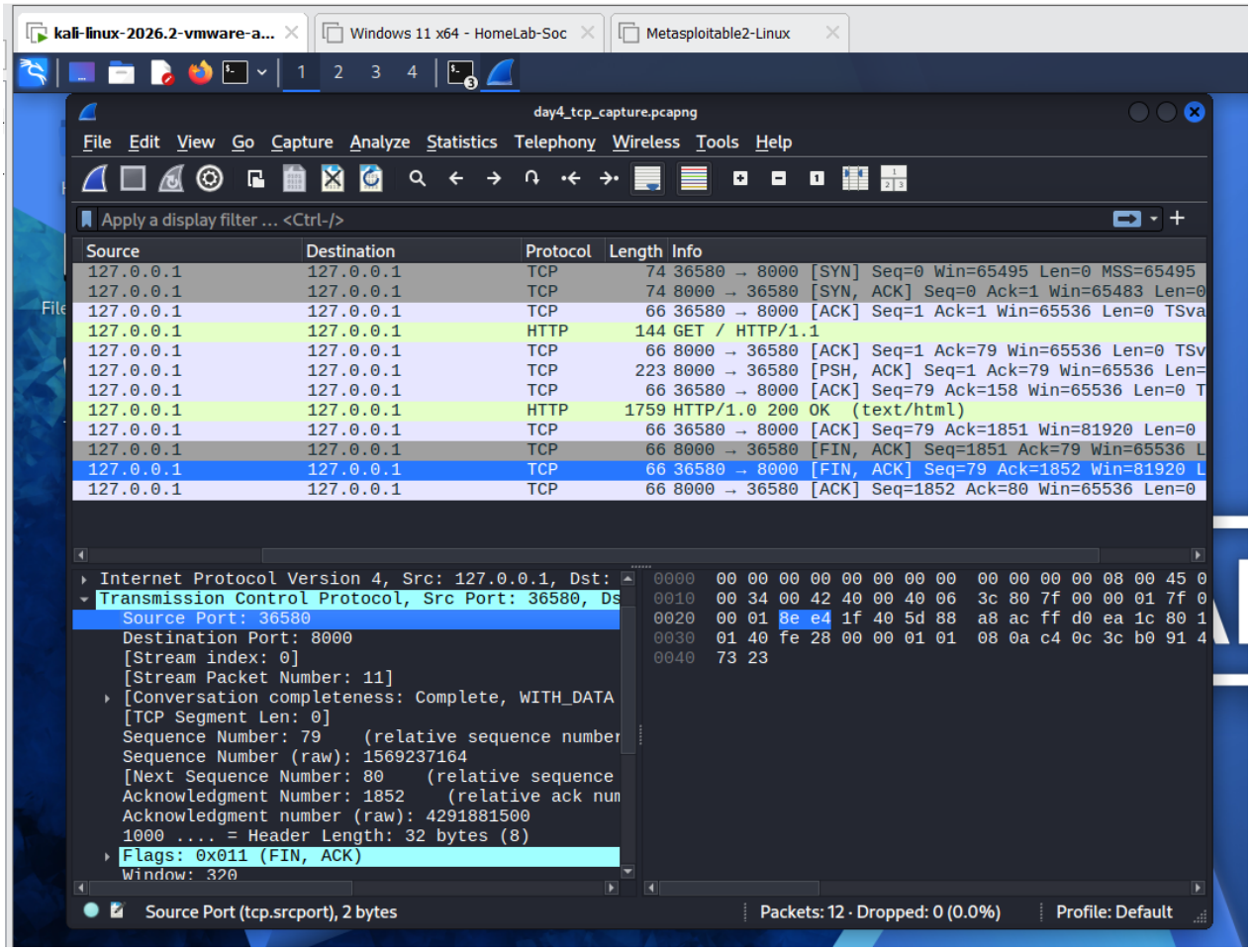
Packet 10: Source Port 8000, Destination Port 36580, Sequence Number 1851, Acknowledgment Number 79, Flags 0x011 (FIN, ACK). The server signals it has no more data to send.



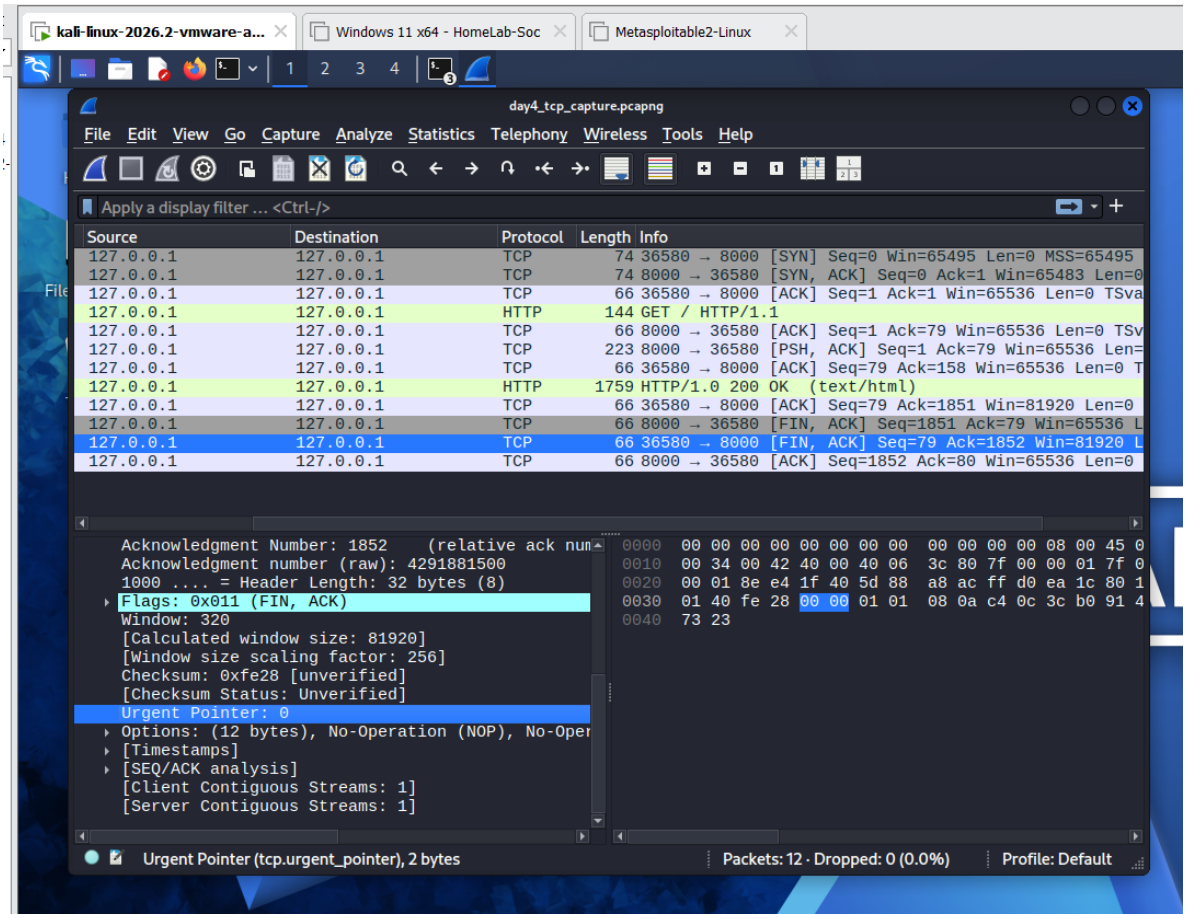
Continued detail for packet 10: the raw acknowledgment number and window scaling confirm the connection's flow-control state was still fully intact at the moment the server chose to initiate closure.

Nothing in this packet's supporting detail suggests an abnormal or forced closure. The advertised window and scaling factor are consistent with the values seen throughout the connection, meaning the server was not closing due to any buffer or flow-control problem, it simply had no more data to send after the HTTP response completed. The Options field again shows timestamp-related entries padded with No-Operation (NOP) bytes, a standard, expected pattern for aligning TCP options to the 4-byte boundaries the protocol requires, not an indication of anything unusual. This is the kind of packet a SOC analyst learns to recognize quickly as a routine, healthy connection close rather than something requiring further investigation.

9.2 Packet 11: FIN, ACK (client response, combined)



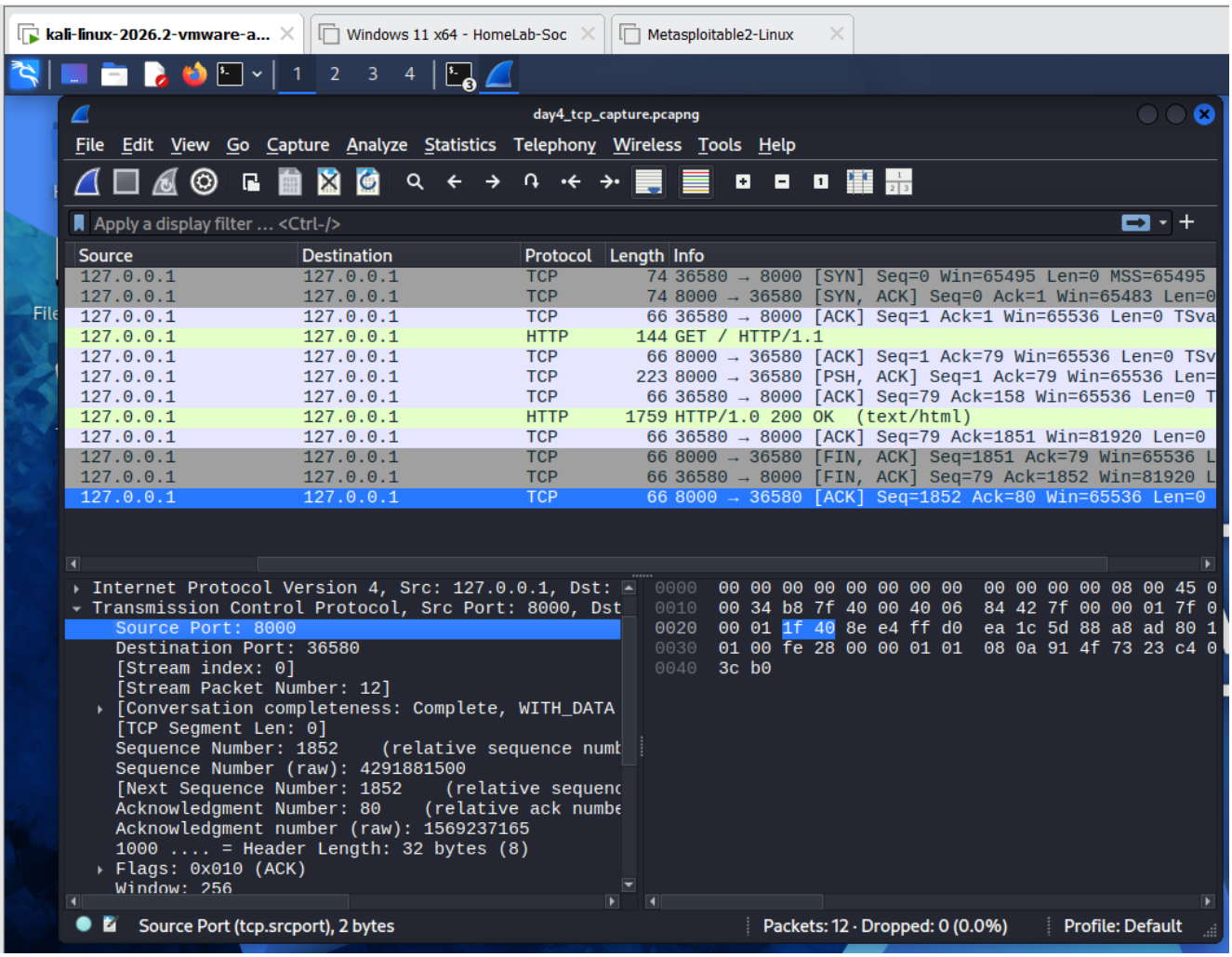
Packet 11: Source Port 36580, Destination Port 8000, Sequence Number 79, Acknowledgment Number 1852, Flags 0x011 (FIN, ACK). The client acknowledges the server's FIN and sends its own FIN in the same packet.



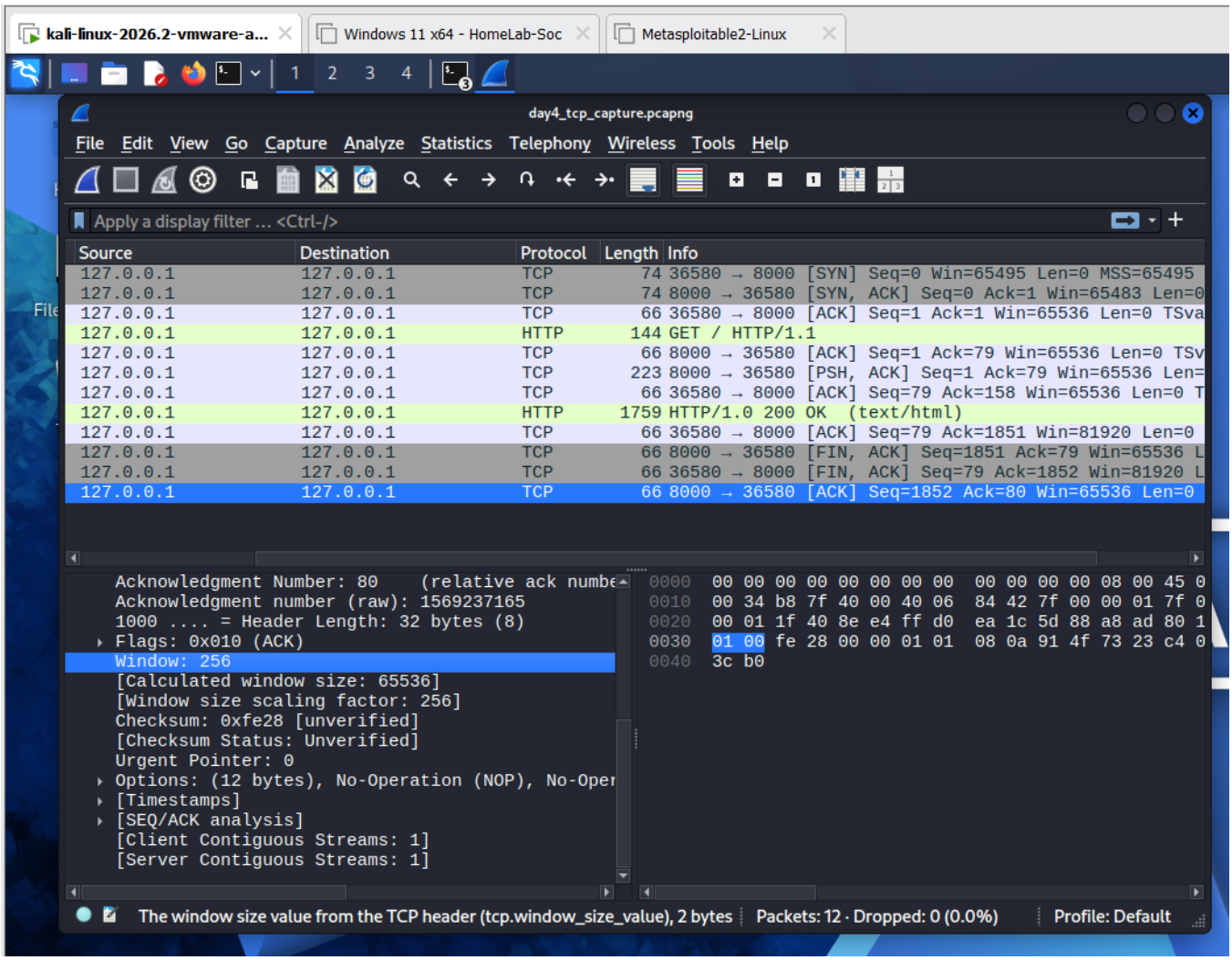
Continued detail for packet 11: the raw acknowledgment number 4291881500 confirms this packet correctly acknowledges the server's FIN at the byte level, alongside carrying the client's own FIN in the same segment.

A FIN is treated by TCP as consuming one sequence number, exactly like a SYN does, even though it carries no payload of its own. The server's raw sequence number in packet 10 was 4291881499, and the client's acknowledgment number here is exactly one higher, at 4291881500, confirming the client correctly accounted for that FIN before adding its own. Worth noting: the advertised window in this packet is larger than in earlier packets (81920 bytes rather than 65536), simply reflecting that the client's receive buffer had more free space available at this exact moment since the HTTP response had already been fully read and processed. This is ordinary window fluctuation, not a signal of anything wrong.

9.3 Packet 12: ACK (final)



Packet 12: Source Port 8000, Destination Port 36580, Sequence Number 1852, Acknowledgment Number 80, Flags 0x010 (ACK). The server acknowledges the client's FIN. The connection is now fully closed.



Continued detail for packet 12: the raw acknowledgment number 1569237165 matches the client's final raw sequence position, confirming the close was acknowledged correctly and no data was lost or left unacknowledged.

This final packet closes out the arithmetic chain that has run through the entire capture. The client's raw sequence number in packet 11 was 1569237164, and the server's acknowledgment number here lands exactly one higher, correctly accounting for the client's FIN.

With this confirmed, every single sequence and acknowledgment number across all 12 packets forms one unbroken, verifiable chain from the opening SYN to this final ACK, with no gaps, no retransmissions, and no out-of-order delivery anywhere in the capture. Wireshark's own stream tracking confirms the same conclusion in its final line: both Client and Server Contiguous Streams read 1, meaning exactly one clean, uninterrupted TCP stream existed for the entire capture, precisely what a textbook connection lifecycle should look like.

10. Connection Summary

Pkt	Direction	Flags	Seq	Ack	Event
1	36580 -> 8000	SYN	0	-	Client opens connection
2	8000 -> 36580	SYN, ACK	0	1	Server accepts and acknowledges
3	36580 -> 8000	ACK	1	1	Client confirms; connection open
4-9	both directions	ACK / PSH, ACK	-	-	HTTP GET request and response
10	8000 -> 36580	FIN, ACK	1851	79	Server signals end of data
11	36580 -> 8000	FIN, ACK	79	1852	Client acknowledges and closes its side too
12	8000 -> 36580	ACK	1852	80	Server acknowledges; connection fully closed

11. Findings

11.1 The teardown closed in three packets, not the textbook four

A TCP close is often taught as four separate packets: FIN from the initiator, ACK from the other side, FIN from the other side, and a final ACK. This capture shows a variation on that pattern. Packet 11 combines the client's acknowledgment of the server's FIN with the client's own FIN in a single packet, rather than sending them separately. This is a normal and common optimization, since there was no additional data the client needed to send between acknowledging and closing its own side, so the two flags were piggybacked onto one packet instead of two. Recognizing this as a valid variation, rather than assuming something was missing, is part of correctly reading real teardown behavior.

11.2 Port usage matched the expected client-server pattern exactly

Across all 12 packets, the server held a single fixed port throughout the entire connection (8000), while the client used one consistent ephemeral port for the life of the connection (36580), assigned automatically by the operating system rather than chosen deliberately. This is the standard, expected pattern: a server advertises a known port so clients can find it, while each client connection gets its own temporary port so the operating system can track multiple simultaneous connections without confusion.

11.3 Capturing on loopback removed every source of unrelated noise

Unlike the Day 3 capture on eth0, which mixed DNS lookups, ARP broadcasts, and background application traffic together with the traffic being investigated, this capture on lo contains exactly 12 packets and nothing else. Every single packet belongs to the one connection under study. This is not an accident of a quiet network, it is a direct result of capturing on the interface that only ever carries traffic between a machine and itself. Choosing the right interface for what is actually being investigated is as much a part of correct analysis as reading the packets themselves.

11.4 Sequence numbers displayed by Wireshark are relative, not the actual values sent

The Sequence Number field shown for packet 1 reads 0, but the raw value carried in the actual packet was 1569237085, a large, effectively random number, visible directly in the raw hex bytes below the field. Wireshark converts these to relative numbers, starting from 0 for the first packet of each direction, purely to make the sequence easier for a human to follow. Real TCP implementations pick unpredictable starting sequence numbers deliberately, as a basic protection against an attacker guessing a connection's sequence numbers and injecting forged packets into it.

12. Conclusion

A single, deliberately isolated TCP connection was captured end to end on the loopback interface, from the opening three-way handshake through data transfer to a three-packet connection teardown. Every packet in the handshake and close was individually selected and its header fields, source and destination ports, sequence numbers, acknowledgment numbers, and flags, confirmed directly rather than assumed from the summary view. The findings go beyond simply locating the required packets: they cover a real variation in standard teardown behavior, the expected asymmetry between fixed server ports and ephemeral client ports, the analytical value of choosing the correct capture interface, and the distinction between Wireshark's relative sequence numbers and the actual randomized values TCP uses in practice. Together this demonstrates not just the ability to find SYN, SYN-ACK, and ACK packets, but a working understanding of why TCP connections are built and closed the way they are.